

7位解法 — 3D nnU-Net + blob regression (再び)

Rank	7
Team	MIC-DKFZ
Author	Stefan Denner
Topic ID	612039
Version	Japanese Translation

Source: <https://www.kaggle.com/competitions/rsna-intracranial-aneurysm-detection/discussion/612039>
Retrieved with Kaggle CLI on 2026-07-07.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

このIntracranial Aneurysm Detection competitionを企画してくださった@evancalabrese、@ryanholbrook、RSNA、Kaggleに感謝します。

概要 (TL;DR)

私たちの解法は単純で実装しやすいものです。

- タスクをmultichannel blob regressionとして定式化し、TopK (20%) BCE lossで最適化した後、チャンネルごとの最大値を確率予測とする。
- 3D medical image segmentationの代表的frameworkであるnnU-Netを土台とする。私たちはすでにBYU - Locating Bacterial Flagellar Motors 2025の2位解法でこれを応用していた。
- modelはresidual encoderを持つ3D U-Netで、scratchから学習した。
- 時間制約のため、推論はTTAなしのsingle modelで行った。
- Public/Private leaderboardのscoreは0.83 / 0.83だった。

私たちについて

私たちはGerman Cancer Research CenterのDivisions of Medical Image ComputingとHelmholtz Imagingに所属する同僚 (scientistとPhD student) のteamである。専門は3D image analysis、特に3D segmentation問題の解決とalgorithmを臨床へ導入するinfrastructureの開発である。

Method

このタスクをheatmap regressionとしてモデル化し、BYU - Locating Bacterial Flagellar Motors 2025の2位解法を発展させた。

Data used

pydicom libraryを使ってDICOM seriesを.nii.gzへ変換し、2D DICOM fileからなるseriesは並列処理した。同じlibraryでspacing、origin、directionも抽出した。その結果をSimpleITK imageへ変換し、RAS orientationへ揃えた。

計算資源を減らすため、imageの中央上部に[200, 160, 160] mmの立方体Region of Interest (ROI) を設定した。training setの全動脈瘤がROI内に入ることを確認した。Figure 1はimage上のROIを示す。

複数のseriesには、想定外のorientation、空のseries、shunt artifact、movement artifact、上部が空のimageなどの欠陥があった。誤ったorientationのような軽度のartifactを持つseriesは保持してflipで修正し、完全に空のimageのような強いartifactを持つseriesは除外した。合計10 volumeを除外した。



Preprocessing

data preprocessingはself-configurable segmentation frameworkのnnU-Netを3D full resolution configurationで用いて行った。volumeはRAS orientationも強制しようとするSimpleITK readerで読み込んだ。全training imageから得たmedian spacing 0.70, 0.47, 0.47へ全imageをresampleし、training datasetから求めたglobal meanとglobal standard deviationに基づくz-score normalizationを適用した。

強い時間制約があったため、`scipy.ndimage.zoom`などの一般的なfunctionより高速なPyTorch製のspecial resamplerを使った。

Network architecture

nnU-NetのResEncを使った。これは本質的にはresidual encoderと軽量のconvolutional decoderを持つUNetである。architectureは6 stageからなり、各stageのfeature数は順に [32, 64, 128, 256, 320, 320] である。

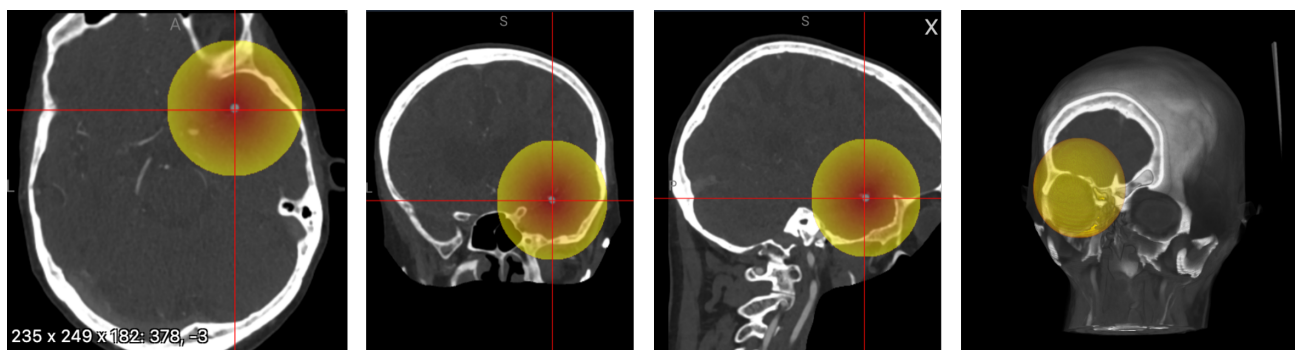
Training procedure

challenge dataを5つのcross-validation foldへ分割した。全image modalityで十分な性能を確保しなかったため、vessel classではなくmodalityでfoldをstratifyした。参戦が比較的遅く、試すべきdesign choiceが多かったため、hyperparameter tuningの大半はcross-validationの最初のfoldだけで行った。

Blob regression with nnU-Net

nnU-Netはsemantic segmentation用に作られており、期待するdata structureもそれに沿っている。動脈瘤regressionへ対応させるため、ground truthをsemantic segmentation mapとして保存した。各動脈瘤は半径 $r=5$ voxelsのsphereで表し、そのinteger labelをground-truth動脈瘤のvessel classとした。nnU-Netはこれらのsphereをsegmentationとして扱い、通常どおりdata loadingとaugmentation pipelineを通すため、rotationやmirroringなども正しく適用される。ただし複数labelにleft/rightの符号化があるため、left/right axisのmirroring augmentationは適用しなかった。dataloading pipelineの末尾にcustom transformを挿入し、各動脈瘤instanceを対応するchannelのblobへ変換した。14 classを別々のchannelとしてmodel化し、第14 class (Aneurysm Present) は13個のanatomical classのpixelwise maximumとした。

「EDT blob」、つまりEuclidean Distance Transform (EDT) を適用し、値域を [0, 1] へrescaleした3D sphereを使った。最初のcross-validation foldで最適化されたsphere sizeは65 voxelsだった。sphere radiusは15から95 voxelsまで実験した。



Hyperparameters

最終modelはbatch size 32、patch size 96x160x128 voxels で学習した。initial learning rateは0.01で、default nnU-Netと同じpolyLR scheduleにより学習中に減衰させた。SGDを使い、3000 epochs（1 epochあたり 250 iterations）学習した。loss functionはbinary cross-entropyで、batch全体から計算したloss値が最大の20%に当たるvoxelだけで計算した。

最終modelは4xA100 40GB上でPyTorchのDDPを用いて学習し、4.5日かった。

Inference

nnU-Netのinference infrastructureをほぼそのまま使った。input seriesを一連のpatchに分割し、各patchでblob regressionを行う。channelごとの最大値をclass probabilityとし、patch間もmax aggregationして最終予測を得た。

予測には2xT4 instanceを使い、各input seriesの予測patchをGPU間へ均等に分割した。常にsingle modelで、ensembleは使っていない。推論には約8時間かった。

Results

残念ながらKaggle platformの不安定さにかなり影響された。

model自体は3000 epochsまで学習を完了したが、成功した提出はdeadlineの3日前に行った1500 epochsまでのcheckpointだけだった（Private/Public LBとも0.83）。その後の提出はmodel weightsしか変わなかったにもかかわらずtimeoutした。

internal validationでは、後期checkpoint、TTA、patchへのGaussian weightingによってさらに改善した可能性が示された（それ以前の提出でも改善を確認した）。

internal performanceは驚くことに0.9まで上昇したが、leaderboardでは再現されなかった。このshiftの原因は分からない。理由の一つは、そうしなければ15分後にerrorが出たため、inferenceをtry/catch block内へ埋め込まざるを得なかったことかもしれない。正確な理由は不明である。

segmentation maskは活用しなかった。学習時のauxiliary outputとして追加すればmodelのlocalizationを助けられた可能性がある。参戦が遅く検証時間がなかったが、他teamはこれによる改善を示した。

うまくいかなかったこと

- 問題をdetectionとしても定式化し、self-configuration detection frameworkのnnDetectionで解こうとした。この方法はPublic LBではここで紹介した解法より良かったが、Private LBとinternal validationでは劣った。
- nnDetectionはcropped data上のinstance segmentation label版で学習した。self-configured Retina U-Net architectureを使い、boxとして符号化した動脈瘤位置と動脈瘤segmentationから学習した。本解法とは異なり、input seriesをisometricな[1.0, 1.0, 1.0] mm空間へresampleし、batch size 4、100 epochs (1 epochあたり2500 iterations) で学習した。lossはbox座標回帰のL1 lossとbox class推定のfocal lossを組み合わせ、initial learning rate 0.001からpolynomial learning rate scheduleを適用し、Nesterov momentum付きSGDを使った。予測boxはintersection over union threshold 0.1のnon-maximum suppressionでpostprocessした。加えて8 TTAとinference patch overlap 0.25で推論できた。
- image処理を速められる可能性から[1.0, 1.0, 1.0] mmのisometric spaceへのresamplingも実装したが、結果が大幅に悪化したため早期に中止した。
- Aneurysm Present classをより良くmodel化するため、binary classを持つ外部動脈瘤dataset (ADAM、Large IA Segmentation dataset、INSTED、Lausanne TOF-MRA Aneurysm Cohort、Royal Brisbane TOFMRA Intracranial Aneurysm Database、Jianxiaokuang aneurysm dataset) とのco-trainingも調べた。後から一部 (ADAM、Large IA Segmentation dataset) が許可されていないと分かったため除外し、それらなしでco-trainingした。結局co-trainingは実質的に効かず、challenge dataだけをscratchから学習する方法へ戻した。
- 最初はimage全体を処理したが、時間制限からROI周辺をcropするようにしたところ、性能も向上した。
- 最終modelではpatch予測を単にmax aggregationした。ただしmodelはedge付近に不確実性を持つことが知られている。一般的な対策は各patch予測へのGaussian weighting (中央を高weight、borderを低weight) で、以前の提出では改善した。しかしplatformの不安定さにより最終modelでは適用できなかった。
- より大きなpatch sizeでも学習したが、意外にもscore改善には寄与しなかった。

望んでいたこと

Discussionでも述べたとおり、predict functionのsignatureによって処理の並列化がかなり制限された。詳細はこちら。

さらに特定のnumpy versionが必要で、codeをsubprocessとして動かさざるを得なかった。最終的にproxy workerを起動し、standard output経由で通信した。このため大量のboilerplateと複雑さが増え、不自然な構成になった。

challenge最終日の数日間teamから送られる提出が増えるにつれて多くの提出がtimeoutしたため、提出時間を長くし、提出数への依存が少ないserver architectureにしてほしかった。

より広い意味では、Kaggleのsubmission notebook方式は今回もかなり難しかった。Docker containerが使えると柔軟性が高く、はるかに容易だっただろう。

Acknowledgements

主催したRSNAとhostしたKaggleに感謝する。またGerman Cancer Research CenterのDivisions of Medical Image ComputingとHelmholtz Imagingにも感謝したい。

補足Q&A

Ma Edward（質問）：

有益なwrite-upを共有していただきありがとうございます。single modelが完全に最適化されていなくてもこれほど高性能なのは印象的です。次の点を教えてください。

1. [200, 160, 160] ROIを自動的にどう求めましたか。
2. APなしのimageは対応するmaskがすべて0になりますが、どう学習に使いましたか。
3. class imbalanceへの対策はありますか。
4. inferenceではTTAを使わなかったのでしょうか。
5. fold 0 splitでsphere sizeの違いは性能へどう影響しましたか。
6. code公開の予定はありますか。

Stefan Denner（回答）：

1. write-upどおり非常に単純で、それぞれのROI sizeをmm単位で指定し、imageの中央上部をcropしただけです。
2. APなし、という意味ですね。その場合target heatmapは全要素が0ですが、問題ありません。
3. class weightingとlow-prevalence class oversamplingをnnDetectionで実装しましたが効果はありませんでした。nnU-Netでは試していません。ただしある特定classは常にかなり低性能だったので試す価値はあります。一方、同じくprevalenceの低い別classは妥当な性能だったため、class prevalenceだけが原因ではありません。
4. そのとおりです。stepsize 0.5とmedian spacingではTTAを使うとinferenceが長すぎました。
5. EDT25 0.888; EDT35 0.876; EDT45 0.893; EDT55 0.883; EDT65 0.896; EDT75 0.871; EDT85 0.871; EDT95 0.866でした。
6. はい、翌週に公開予定です。